KHOA ĐIỆN TỬ VIỄN THÔNG

LẬP TRÌNH ĐA NỀN TẢNG

Chủ đề 5. Danh sách, lưới dữ liệu và cuộn

Flutter: ListView • GridView • Scroll • Lazy loading

Sinh viên thực hiện:
Trần Đăng Duy
Phan Văn Trà

Giảng viên hướng dẫn:
Nguyễn Duy Nhật Viễn

Nội dung trình bày

- **1. Cơ sở lý thuyết**
  - Cơ chế cuộn: Viewport & Sliver, nguyên lý bố cục (constraints)
  - Dựng lười: Widget – Element – RenderObject; ListView, GridView
  - ScrollController, ScrollPhysics, lazy loading & phân trang
  - Cuộn lồng nhau, SingleChildScrollView, shrinkWrap
  - Item kích thước khác nhau, trạng thái rỗng, RefreshIndicator
- **2. Thực hành – demo ứng dụng "Thư viện mini"**
  - Chức năng ⇄ kiến thức & cấu trúc project
  - Cuộn lồng nhau, lazy loading, tìm kiếm/rỗng, item co giãn
  - GridView & trang chi tiết (SingleChildScrollView)
- **3. Kết luận & tài liệu tham khảo**

TỔ CHỨC THỰC HIỆN

Phân công công việc

Thành viên	Nhiệm vụ thực hiện	Tỷ lệ
Phan Văn Trà	Thiết kế mô hình sách và ánh xạ dữ liệu CSV: ISBN, tác giả, thể loại, mô tả và thông tin xuất bản. Xây dựng các màn hình danh sách, lưới, chi tiết và điều hướng giữa các tab. Xử lý tìm kiếm trên giao diện, phân trang hiển thị, làm mới và trạng thái tải, lỗi, rỗng.	**50%**
Trần Đăng Duy	Đọc, kiểm tra và phân tích CSV; cache dữ liệu, làm mới và lọc sách theo tên hoặc tác giả. Xây dựng thẻ sách, mục danh sách, ảnh bìa và giao diện thông báo rỗng. Xử lý ảnh đang tải hoặc lỗi và thao tác mở rộng, thu gọn mô tả sách.	**50%**

Cơ chế cuộn trong Flutter: Viewport & Sliver

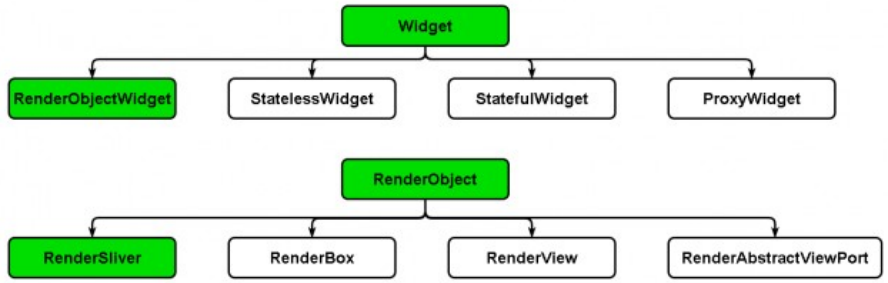
ListView, GridView và CustomScrollView kết hợp Scrollable với viewport để tổ chức nội dung cuộn.

Scrollable tiếp nhận thao tác và quản lý vị trí cuộn. Viewport là khung hiển thị một phần nội dung.

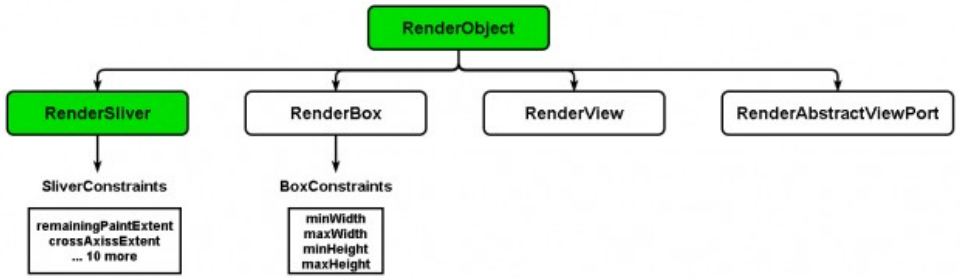
Viewport nhận các sliver. SliverConstraints truyền ràng buộc; SliverGeometry mô tả kết quả bố cục.

SliverList/SliverGrid với builder tạo mục theo nhu cầu. Không phải mọi sliver đều tự tạo nội dung theo cách này.

ListView thường tương ứng SliverList, GridView tương ứng SliverGrid. SliverToBoxAdapter đưa một widget dạng box vào vùng sliver.



Widget / RenderObject: RenderSliver bên cạnh RenderBox



RenderSliver dùng SliverConstraints thay cho BoxConstraints

Nguyên lý bố cục: ràng buộc đi xuống, kích thước đi lên

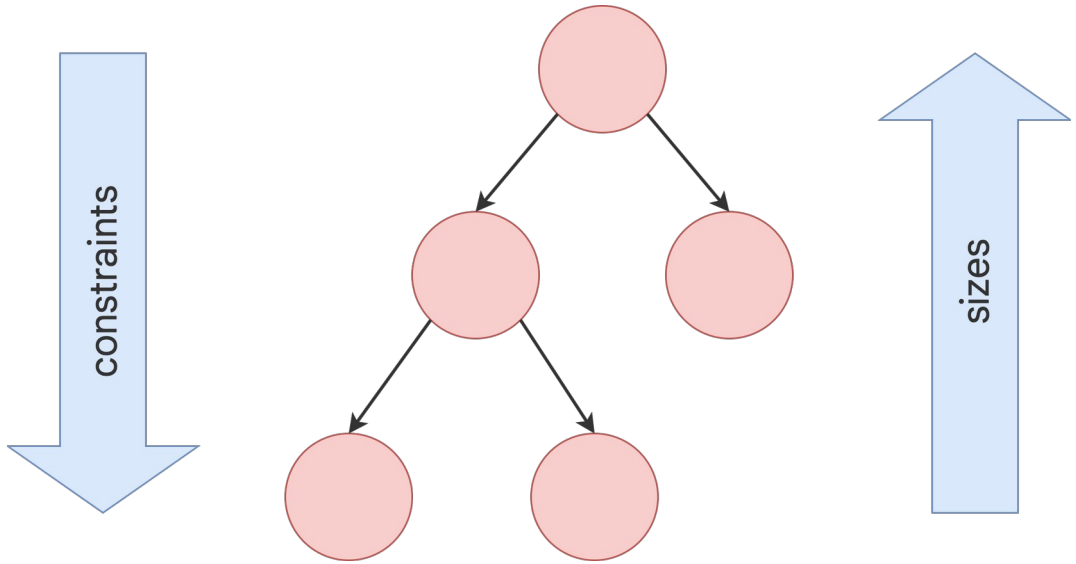
Cha truyền ràng buộc xuống. Con chọn kích thước trong giới hạn cho phép. Cha đặt vị trí của con.

Viewport thường cần kích thước hữu hạn. Nội dung bên trong có thể dài hơn khung hiển thị. Hai khái niệm này cần phân biệt.

ListView dọc đặt trực tiếp trong Column có thể nhận chiều cao không giới hạn và gây lỗi viewport.

Dùng Expanded khi Column cha có chiều cao hữu hạn, hoặc SizedBox với chiều cao phù hợp. Expanded không giải quyết được mọi trường hợp lồng trong vùng cuộn.

Ràng buộc gồm min/max chiều rộng và chiều cao. Với vùng cuộn dọc, phải xác định chiều cao viewport; không ép toàn bộ nội dung vào chiều cao đó.



"Constraints go down. Sizes go up. Parent sets position."

Cơ chế tạo phần tử theo nhu cầu trong Flutter

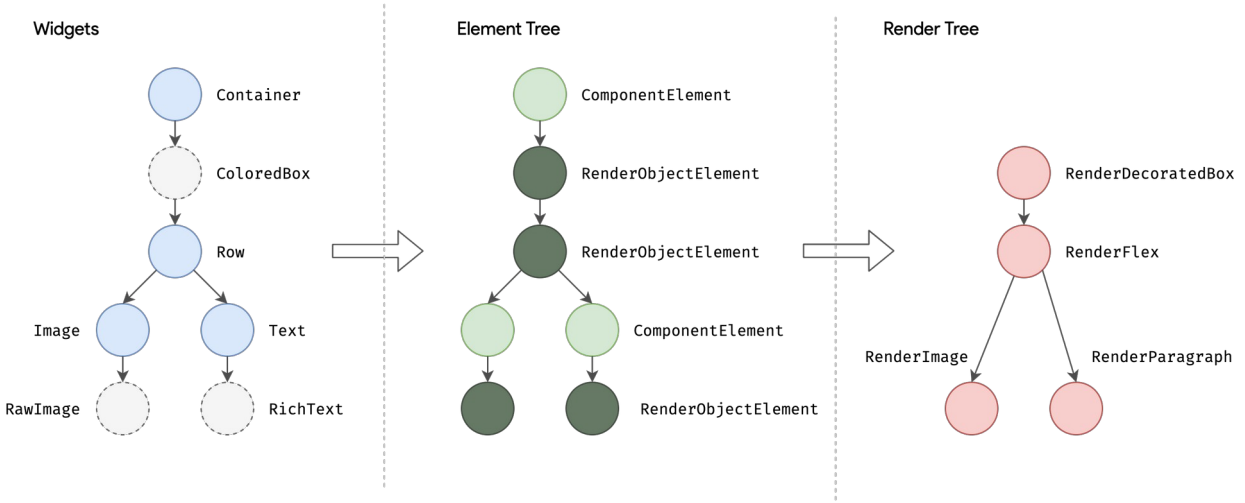
Widget mô tả cấu hình. **Element** gắn widget với vị trí và vòng đời trong cây. **RenderObject** thực hiện bố cục và vẽ.

Builder tạo mục khi cần cho viewport và vùng cache. Mục ở xa có thể bị hủy, trừ các trường hợp được giữ lại bằng keep-alive.

Key giúp nhận diện mục khi dữ liệu đổi vị trí. Keep-alive giữ một số mục ngoài vùng nhìn thấy, nên lượng bộ nhớ không chỉ phụ thuộc số mục đang hiện.

Khi đối chiếu widget ở vị trí tương ứng, Flutter có thể cập nhật Element nếu runtimeType và key phù hợp.

Trạng thái cần giữ lâu, như sách yêu thích hoặc mô tả đang mở, nên gắn với ID sách và lưu ở nơi phù hợp. Số mục còn sống còn phụ thuộc cache và keep-alive.



Cây Widget → Element → Render (tài liệu Flutter)

ListView: dựng toàn bộ so với dựng lười

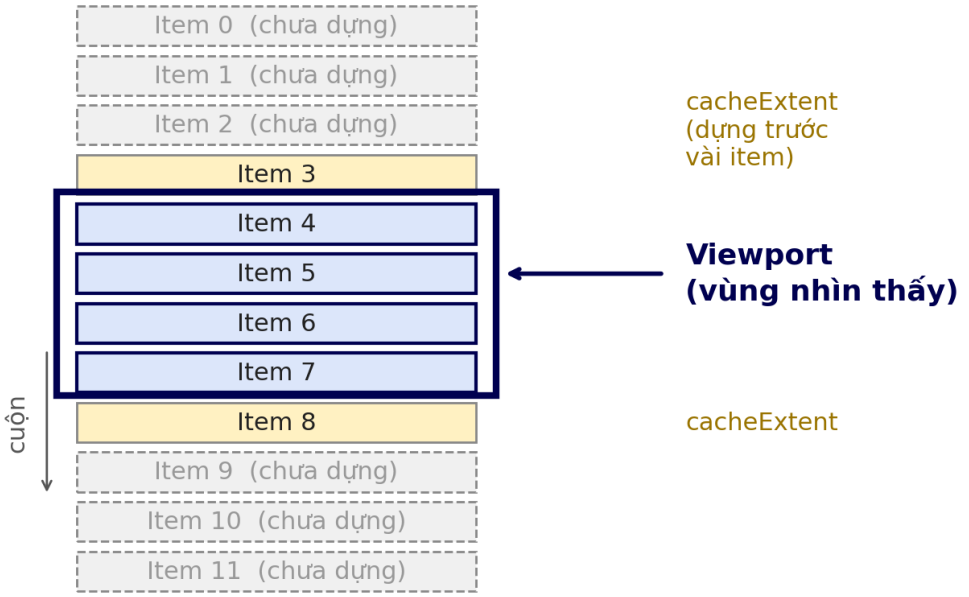
ListView(children: ...) nhận danh sách widget đã tạo sẵn. Cách này phù hợp khi có ít mục.

ListView.builder gọi itemBuilder để tạo mục theo nhu cầu cho viewport và vùng đệm.

itemCount không bắt buộc. Khi biết số mục, nên cung cấp để hỗ trợ ước lượng phạm vi cuộn.

ListView.separated tạo thêm phần phân cách. scrollDirection chọn cuộn dọc hoặc ngang.

Mục ngoài vùng dựng có thể bị hủy. Trạng thái cần tồn tại lâu nên lưu theo ID. ListView.separated tạo phần ngăn cách giữa các mục theo nhu cầu.



Chỉ item trong viewport (xanh) và vùng đệm (vàng) được dựng

GridView: nguyên lý chia ô qua SliverGridDelegate

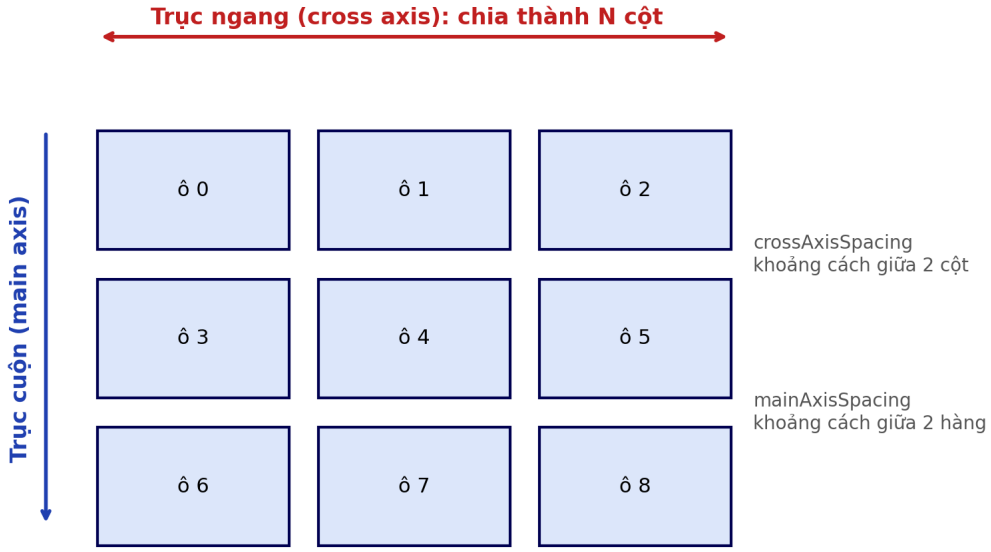
GridView trình bày dữ liệu hai chiều. GridView.builder tạo các widget ô theo nhu cầu.

Với lưới cuộn dọc, FixedCrossAxisCount cố định số cột. MaxCrossAxisExtent đặt độ rộng tối đa của ô.

Kích thước ô tính trên phần bề rộng còn lại sau padding và khoảng cách giữa các cột.

childAspectRatio hoặc mainAxisExtent quy định kích thước ô với các delegate chuẩn. Lưới này phù hợp ảnh bìa và tiêu đề giới hạn dòng.

Khi cuộn ngang, trục chính là ngang và số ô theo trục chéo là số hàng. Với cuộn dọc, chiều cao ô bằng độ rộng chia childAspectRatio, trừ khi dùng mainAxisExtent.



ScrollController, ScrollPosition & ScrollPhysics

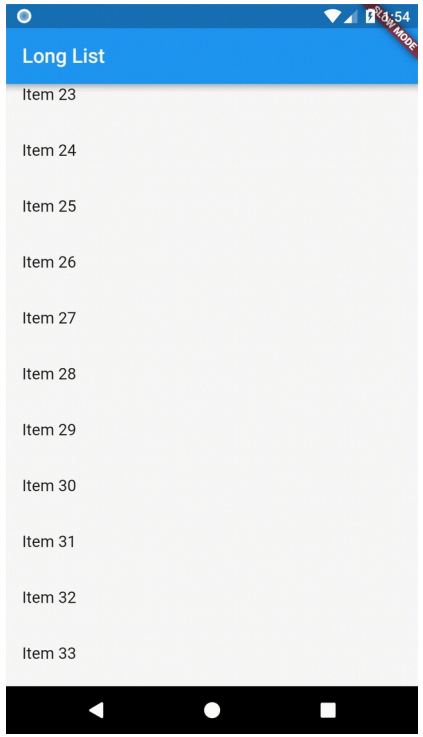
ScrollPosition cung cấp pixels, minScrollExtent, maxScrollExtent và viewportDimension. extentAfter cho biết phần nội dung còn lại sau viewport.

ScrollController đọc vị trí, lắng nghe cuộn hoặc điều khiển bằng jumpTo và animateTo. Kiểm tra hasClients trước khi truy cập position.

ScrollPhysics quy định phản hồi cuộn: Clamping, Bouncing, NeverScrollable hoặc AlwaysScrollable.

NotificationListener<ScrollNotification> là cách khác để nhận thông báo cuộn. Controller do State tạo cần được dispose khi không còn dùng.

jumpTo chuyển vị trí ngay; animateTo tạo chuyển động theo thời gian. Chỉ đọc phạm vi cuộn khi vị trí đã gắn và có thông tin kích thước nội dung.



Danh sách dài cuộn trong viewport (cookbook)

Lazy loading (infinite scroll) & phân trang

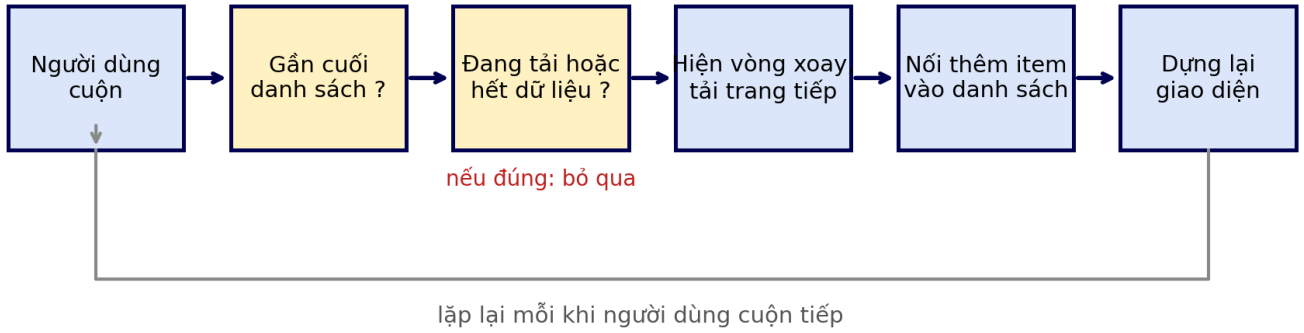
Tạo widget theo nhu cầu: builder giảm việc tạo sẵn giao diện cho các mục chưa cần hiển thị.

Trong Thư viện mini, CSV có 200 sách. Mỗi đợt mở thêm 20 kết quả. Future.delayed chỉ mô phỏng thời gian chờ.

Nếu dùng API, trang tiếp theo có thể xác định bằng offset/limit hoặc cursor. Với dữ liệu cục bộ của app, phân trang chỉ tăng số kết quả hiển thị.

Phân trang: lấy hoặc mở thêm từng nhóm kết quả. Có thể dùng với API hoặc dữ liệu cục bộ đã nạp sẵn.

Khi gần cuối, kiểm tra đang tải và còn kết quả trước khi mở đợt tiếp theo. Cần phân biệt đang tải, còn dữ liệu và đã hết dữ liệu.



Luồng tải thêm khi cuộn

Cuộn lồng nhau, SingleChildScrollView & shrinkWrap

SingleChildScrollView cuộn một widget con, thường là Column. Phù hợp trang chi tiết có số khối nội dung ít.

Khác hướng: dải sách ngang trong danh sách dọc cần chiều cao hữu hạn, chẳng hạn dùng SizedBox.

Cùng hướng: nếu muốn nhiều khối cuộn chung, dùng CustomScrollView với SliverList, SliverGrid và SliverToBoxAdapter.

shrinkWrap: true xác định kích thước vùng cuộn theo nội dung, có thể tốn chi phí tính lại. Không thể khẳng định mọi bố cục dùng shrinkWrap đều dựng hết mục hoặc luôn mất lazy building.

Cuộn cùng hướng không phải luôn sai. Nếu cần các vùng cuộn độc lập phải giới hạn kích thước. Nếu cần một luồng cuộn chung, tổ chức bằng sliver.

Item kích thước khác nhau, trạng thái rỗng & kéo để làm mới

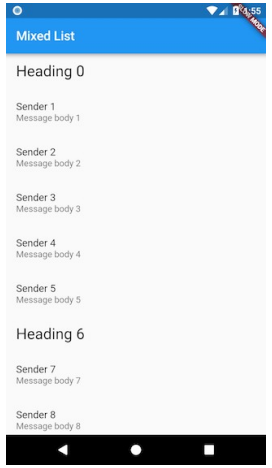
Với mô tả mở rộng, để mục tự xác định chiều cao. Không ép mọi mục vào cùng itemExtent.

Nếu các mục có chiều cao đồng nhất, itemExtent hoặc prototypeItem giúp cơ chế cuộn biết trước chiều dài mục và giảm công việc tính toán.

Khi không có kết quả, hiện lý do và hành động như xóa tìm kiếm. Phân biệt dữ liệu rỗng với đang tải và lỗi đọc file.

RefreshIndicator dùng với vùng cuộn dọc. onRefresh trả về Future. AlwaysScrollableScrollPhysics hỗ trợ kéo làm mới cả khi nội dung ngắn.

Không đọc CSV hoặc xử lý nặng trong itemBuilder. Dùng ảnh phù hợp, hạn chế rebuild không cần thiết và kiểm tra thời gian build/raster bằng DevTools ở profile mode.



Danh sách có nhiều kiểu item (cookbook)



RefreshIndicator – kéo để làm mới

App thư viện mini

Chức năng trong app	Kiến thức minh họa
2 tab: Danh sách / Lưới	BottomNavigationBar, IndexedStack
Danh sách sách + tải thêm khi cuộn	ListView.builder, ScrollController
“Sách nổi bật” cuộn ngang trong danh sách dọc	Cuộn lồng nhau
Mô tả dài/ngắn, “Xem thêm / Thu gọn”	Phần tử kích thước khác nhau
Tìm kiếm; không kết quả → EmptyView; kéo để làm mới	Trạng thái rỗng, RefreshIndicator
Lưới sách tự chia cột	GridView.builder
Trang chi tiết sách	SingleChildScrollView

Dữ liệu: assets/data/books_200.csv (200 cuốn) đọc bằng BookRepository (rootBundle + gói csv, cache Future). Không dùng DB/API, chỉ dùng setState.

ListView.builder và cuộn lồng nhau

Tạo mục theo chỉ số

```
itemCount: prefixCount + visible + 1,
itemBuilder: (BuildContext context, int index) {
  if (showFeatured && index == 0) {
    return _FeaturedSection(
      books: _allBooks.take(10).toList(growable: false),
      onBookTap: _openDetail);
  }
  final int bookIndex = index - prefixCount;
  if (bookIndex < visible) {
    final Book book = results[bookIndex];
    return BookTile(
      key: ValueKey<String>(book.isbn13),
      book: book,
      onTap: () => _openDetail(book),
    );
  }
```

itemCount: dải nổi bật nếu có, số sách đang hiển thị và một ô trạng thái cuối.

bookIndex: trừ vị trí dải nổi bật để lấy đúng sách trong results.

Cuộn lồng nhau: _FeaturedSection chứa ListView ngang trong vùng cao 220. ValueKey dùng ISBN để nhận diện sách.

Tải thêm khi cuộn gần cuối

Hàm _loadMore

```
Future<void> _loadMore() async {
  if (_isLoadingMore ||
      _visibleCount >= _results.length) return;
  final int generation = _generation;
  setState(() => _isLoadingMore = true);
  await Future<void>.delayed(
    const Duration(milliseconds: 350));
  if (!mounted || generation != _generation) return;
  setState(() {
    _visibleCount = min(
      _visibleCount + _pageSize, _results.length);
    _isLoadingMore = false;
  });
}
```

Kích hoạt: _onScroll gọi hàm khi còn cách cuối danh sách khoảng 200 px.

Chặn tải trùng: đang tải hoặc đã hết kết quả thì return. Mỗi đợt tăng tối đa 20 sách.

Chờ 350 ms: chỉ mô phỏng. mounted và _generation ngăn cập nhật màn hình đã đóng hoặc kết quả cũ.

Mục co giãn và trạng thái rỗng

Mô tả mở rộng trong BookTile

```
Text(
  book.description,
  maxLines: _expanded ? null : 2,
  overflow: _expanded
      ? TextOverflow.visible
      : TextOverflow.ellipsis,
),
TextButton(
  onPressed: () =>
      setState(() => _expanded = !_expanded),
  child: Text(_expanded ? 'Thu gọn' : 'Xem thêm'),
),
```

Nhánh không có kết quả trong _buildBody

```
if (results.isEmpty) {
  return _statusList(
    message: 'Không tìm thấy sách',
    action: 'Xóa tìm kiếm',
    onPressed: _clearQuery,
  );
}
```

maxLines: hai dòng khi thu gọn, không giới hạn khi mở. Chiều cao mục thay đổi theo nội dung.

setState: đảo _expanded và cập nhật giao diện của mục sách.

results.isEmpty: hiện thông báo cùng nút xóa tìm kiếm. _statusList còn hỗ trợ kéo làm mới.

GridView.builder và grid delegate

Thông số lưới trong GridView.builder

```
gridDelegate:
  SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: columns,
    mainAxisExtent: 250,
    crossAxisSpacing: 8,
    mainAxisSpacing: 8,
  ),
itemCount: books.length,
```

Tính số cột trong LayoutBuilder

```
final int columns = constraints.maxWidth < 350
    ? 1
    : max(2,
        ((constraints.maxWidth + 8) / 190).floor());
```

columns: dưới 350 dùng một cột; từ 350 trở lên có ít nhất hai cột.

mainAxisExtent: ô cao 250. Hai thông số spacing tạo khoảng cách 8 giữa các ô.

GridView.builder: tạo ô theo nhu cầu. itemBuilder của app trả về BookCard và mở trang chi tiết khi chạm.

SingleChildScrollView ở trang chi tiết

Khung nội dung của BookDetailScreen

```
body: SingleChildScrollView(
  child: Center(
    child: ConstrainedBox(
      constraints: const BoxConstraints(maxWidth: 720),
```

Khối ảnh bìa bên trong nội dung

```
SizedBox(
  width: 180,
  height: 250,
  child: BookCover(book: book),
),
```

Mô tả sách trong Column

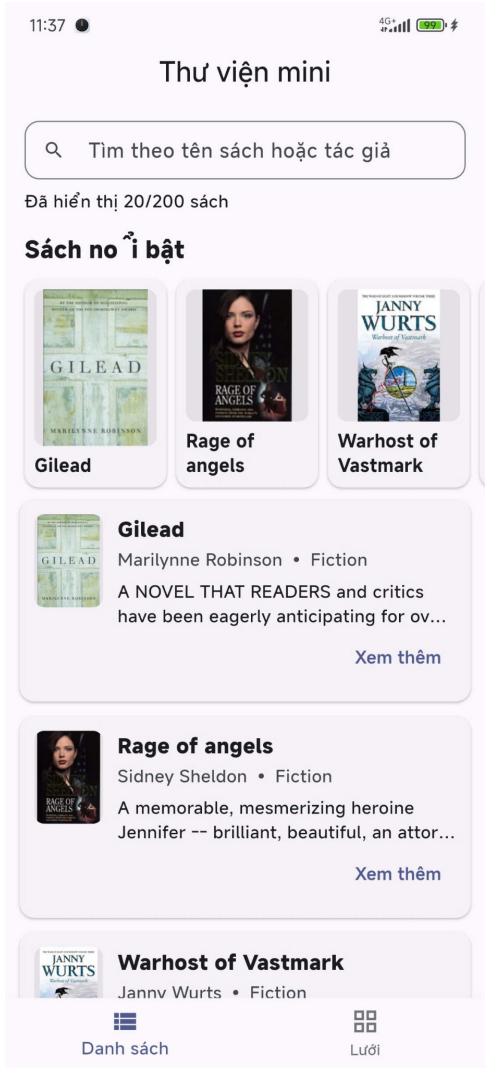
```
Text(book.description,
    style: Theme.of(context).textTheme.bodyLarge),
```

SingleChildScrollView: cuộn cả khối ảnh và thông tin khi nội dung vượt chiều cao màn hình.

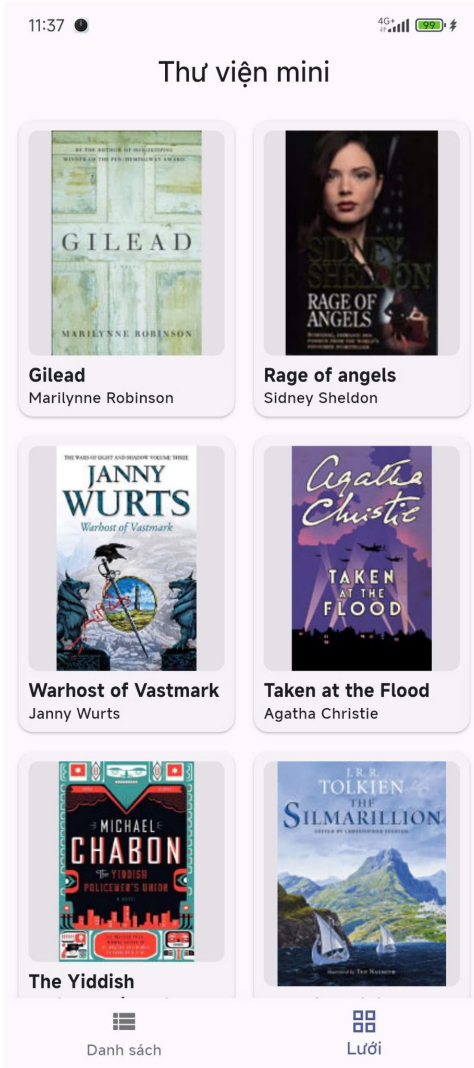
ConstrainedBox: giới hạn nội dung rộng tối đa 720, giúp đọc trên desktop.

Column: chứa các khối nội dung. Dùng cho một trang chi tiết có số khối ít, thay vì danh sách dài.

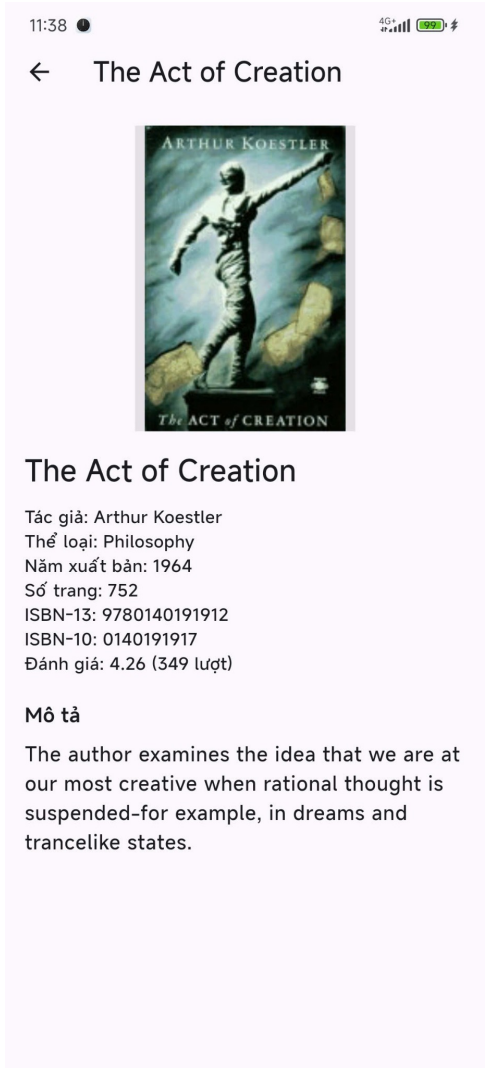
Demo sản phẩm: App thư viện mini



Danh sách và sách nổi bật



Lưới sách



Thông tin chi tiết sách

Kết luận & tài liệu tham khảo

Chọn widget: SingleChildScrollView cho một khối nội dung. ListView.builder và GridView.builder tạo mục theo nhu cầu.

Hiệu năng: giới hạn viewport, tránh xử lý nặng trong build, dùng ảnh phù hợp. Đo bằng DevTools, không kết luận chỉ từ số lượng mục.

Phân trang: app đọc CSV một lần và tăng số sách hiển thị từng đợt. Có kiểm tra mounted và _generation để bỏ kết quả cũ.

Tài liệu tham khảo: Flutter Docs: Lists, Understanding constraints, Architectural overview, Performance best practices. Flutter API: ListView, GridView, SingleChildScrollView, ScrollController, RefreshIndicator, shrinkWrap.

CẢM ƠN THẦY VÀ CÁC BẠN

Xin mời đặt câu hỏi – Q&A

Trần Đăng Duy • Phan Văn Trà